```
//統資二甲 陳彥嘉 412421446

//CH1.7 (1) 判斷是否為奇異矩陣

package w1;

public class W1 {

	public static void main(String[] args) {

		int[][] matrix = { { 2, 1 }, { -2, 3 } };

		int determinant = matrix[0][0] * matrix[1][1] - matrix[0][1] * matrix[1][0];

		if (determinant == 0) {

			System.out.println("因為 det is not 0 ,The matrix is singular.");

		} else {

			System.out.println("因為 det!=0 ,The matrix is nonsingular.");

		}

	}

}
```

```
//CH1.7 (3) 判斷是否為奇異矩陣 如果不是 找出反矩陣

package w1;

public class W1 {

  public static void main(String[] args) {

    int[][] matrix = { { 1, 1 }, { 3, 4 } };

    int determinant = matrix[0][0] * matrix[1][1] - matrix[0][1] * matrix[1][0];

    if (determinant == 0) {

      System.out.println("The matrix is singular.");

    } else {

      System.out.println("The matrix is nonsingular.");
```

```
      double[][] inverse = new double[2][2];

      inverse[0][0] = matrix[1][1] / (double) determinant;

      inverse[0][1] = -matrix[0][1] / (double) determinant;

      inverse[1][0] = -matrix[1][0] / (double) determinant;

      inverse[1][1] = matrix[0][0] / (double) determinant;

      System.out.println("A inverse = ");

      for (int i = 0; i < 2; i++) {

        for (int j = 0; j < 2; j++) {

          System.out.print(inverse[i][j] + " ");

        }

        System.out.println();

      }

    }

  }

}
```

//CH1.7 (13) 找出反矩陣

```
package w1;

public class W1 {

        public static void main(String[] args) {

                double[][] A_inverse = { { 2, 3 }, { 1, 4 } };

                double determinant_inverse = A_inverse[0][0] * A_inverse[1][1] -
A_inverse[0][1] * A_inverse[1][0];

                if (determinant_inverse == 0) {
```

```
                System.out.println("沒有反矩陣(det is 0).");
            } else {
                double determinant = 1.0 / determinant_inverse;
                double[][] A = new double[2][2];
                A[0][0] = A_inverse[1][1] * determinant;
                A[0][1] = -A_inverse[0][1] * determinant;
                A[1][0] = -A_inverse[1][0] * determinant;
                A[1][1] = A_inverse[0][0] * determinant;
                System.out.println("Matrix A = ");
                for (int i = 0; i < 2; i++) {
                    for (int j = 0; j < 2; j++) {
                        System.out.print(A[i][j] + " ");
                    }
                    System.out.println();
                }
            }
        }
}
```

```
//CH1.7 (21)判斷當未知數等於多少  此 homogeneous system 會有 nontrivial sol
package w1;
public class W1 {
    public static void main(String[] args) {
```

```
        System.out.println("when");
        for (int lambda = -10; lambda <= 10; lambda++) {
            int[][] A = { { lambda - 1, 2 }, { 2, lambda - 1 } };
            int determinant = A[0][0] * A[1][1] - A[0][1] * A[1][0];
            if (determinant == 0) {
                System.out.println("lambda = " + lambda);
            }
        }
        System.out.println("the homogeneous system have a nontrivial sol");
    }
}
```

```
//CH1.7 (25)solve Ax=B for x
package w1;
public class W1 {
    public static void main(String[] args) {
        double[][] A_inverse = { { 2, 3 }, { 4, 1 } };
        double[] B = { 5, 3 };
        double[] x = new double[2];
        x[0] = A_inverse[0][0] * B[0] + A_inverse[0][1] * B[1];
        x[1] = A_inverse[1][0] * B[0] + A_inverse[1][1] * B[1];
        System.out.println("Solution x:");
        for (int i = 0; i < x.length; i++) {
            System.out.println("x[" + i + "] = " + x[i]);
```

```
        }
    }
}
```

//CH3.1(5a) 用餘因子找 det(A)

```
package w1;
public class W1 {
    public static void main(String[] args) {
        int[][] A = {
                {2, -1},
                {3, 2}
            };
            int determinant = A[0][0] * A[1][1] - A[0][1] * A[1][0];
            System.out.println("Determinant of A: " + determinant);
        }
    }
```

//CH3.1(5b)(5c、15、17 同理 改數字即可) 用餘因子找 det(A)

```
package w1;
public class W1 {
    public static void main(String[] args) {
        int[][] A = {
                {0, 3, 0},
                {2, 0, 0},
                {0, 0, -5}
```

```
            };
            int determinant = A[0][0] * (A[1][1] * A[2][2] - A[1][2] * A[2][1])
                                - A[0][1] * (A[1][0] * A[2][2] - A[1][2] * A[2][0])
                                + A[0][2] * (A[1][0] * A[2][1] - A[1][1] * A[2][0]);
            System.out.println("Determinant of A: " + determinant);
        }
    }
```

//CH3.2(1)(3、5 同理) 找 A 矩陣的所有餘因子

```
package w1;
public class W1 {
    public static void main(String[] args) {
        int[][] A = {
            {1, 0, -2},
            {3, 1, 4},
            {5, 2, -3}
        };
        int[][] cofactors = new int[3][3];
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                cofactors[i][j] = calculateMinor(A, i, j) * ((i + j) % 2 == 0 ?
1 : -1);
            }
        }
```

```
        System.out.println("Cofactor matrix:");
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                System.out.print(cofactors[i][j] + " ");
            }
            System.out.println();
        }
    }
    public static int calculateMinor(int[][] matrix, int row, int col) {
        int[] minor = new int[4];
        int index = 0;
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                if (i != row && j != col) {
                    minor[index++] = matrix[i][j];
                }
            }
        }
        return minor[0] * minor[3] - minor[1] * minor[2];
    }
}
```

//CH3.2(9) 找 adj(A)

```
package w1;
```

```
public class W1 {
    public static void main(String[] args) {
        int[][] A = { { 6, 2, 8 }, { -3, 4, 1 }, { 4, -4, 5 } };
        int[][] cofactors = new int[3][3];
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                cofactors[i][j] = calculateMinor(A, i, j) * ((i + j) % 2 == 0 ? 1 : -1);
            }
        }
        int[][] adjA = new int[3][3];
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                adjA[i][j] = cofactors[j][i];
            }
        }
        System.out.println("Adjugate matrix (adj(A)):");
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                System.out.print(adjA[i][j] + " ");
            }
            System.out.println();
        }
    }
    public static int calculateMinor(int[][] matrix, int row, int col) {
```

```
            int[] minor = new int[4];
            int index = 0;
            for (int i = 0; i < 3; i++) {
                for (int j = 0; j < 3; j++) {
                    if (i != row && j != col) {
                        minor[index++] = matrix[i][j];
                    }
                }
            }
            return minor[0] * minor[3] - minor[1] * minor[2];
        }
}
```

//CH3.2(11c)(ab 同理) 用 adj(A)找 A inverse matrix

```
package w1;
public class W1 {
                    public static void main(String[] args) {
            int[][] A = {
                    {4, 0, 2},
                    {0, 3, 4},
                    {0, 1, -2}
                };
                int[][] cofactors = new int[3][3];
                for (int i = 0; i < 3; i++) {
```

```
        for (int j = 0; j < 3; j++) {
            cofactors[i][j] = calculateMinor(A, i, j) * ((i + j) % 2 == 0 ?
1 : -1);
        }
    }

    int[][] adjA = new int[3][3];
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            adjA[i][j] = cofactors[j][i];
        }
    }

    int detA = 0;
    for (int j = 0; j < 3; j++) {
        detA += A[0][j] * cofactors[0][j];
    }

    if (detA == 0) {
        System.out.println("Matrix A is not invertible.");
        return;
    }

    double[][] inverseA = new double[3][3];
```

```
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                inverseA[i][j] = adjA[i][j] / (double) detA;
            }
        }

        System.out.println("Inverse of A:");
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                System.out.printf("%.2f ", inverseA[i][j]);
            }
            System.out.println();
        }
    }

    public static int calculateMinor(int[][] matrix, int row, int col) {
        int[] minor = new int[4];
        int index = 0;

        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                if (i != row && j != col) {
                    minor[index++] = matrix[i][j];
                }
```

```
                }
            }

            return minor[0] * minor[3] - minor[1] * minor[2];
        }
    }
```

```
//CH3.2(21)(23 同理) 用 cramer's rule 解 x、y、z、w
package w1;
public class W1 {
    public static void main(String[] args) {
        int[][] coefficients = {
                {1, 1, 1, -2},
                {0, 2, 1, 3},
                {2, 1, -2, 2},
                {1, -1, 0, 1}
            };
            int[] constants = {-4, 4, 5, 4};
            int detA = determinant4x4(coefficients);
            if (detA == 0) {
                System.out.println("The system has no unique solution.");
                return;
            }
            int[][] xMatrix = replaceColumn(coefficients, constants, 0);
```

```
        int[][] yMatrix = replaceColumn(coefficients, constants, 1);
        int[][] zMatrix = replaceColumn(coefficients, constants, 2);
        int[][] wMatrix = replaceColumn(coefficients, constants, 3);
        double x = determinant4x4(xMatrix) / (double) detA;
        double y = determinant4x4(yMatrix) / (double) detA;
        double z = determinant4x4(zMatrix) / (double) detA;
        double w = determinant4x4(wMatrix) / (double) detA;
        System.out.println("Solution:");
        System.out.printf("x = %.2f%n", x);
        System.out.printf("y = %.2f%n", y);
        System.out.printf("z = %.2f%n", z);
        System.out.printf("w = %.2f%n", w);
    }
    public static int determinant4x4(int[][] matrix) {
        int det = 0;
        for (int col = 0; col < 4; col++) {
            det += matrix[0][col] * cofactor(matrix, 0, col);
        }
        return det;
    }
    public static int cofactor(int[][] matrix, int row, int col) {
        int[][] minor = new int[3][3];
        int minorRow = 0, minorCol;
        for (int i = 0; i < 4; i++) {
```

```
                if (i == row) continue;
                minorCol = 0;
                for (int j = 0; j < 4; j++) {
                    if (j == col) continue;
                    minor[minorRow][minorCol++] = matrix[i][j];
                }
                minorRow++;
            }
            return determinant3x3(minor) * ((row + col) % 2 == 0 ? 1 : -1);
        }
        public static int determinant3x3(int[][] matrix) {
            return matrix[0][0] * (matrix[1][1] * matrix[2][2] - matrix[1][2] *
matrix[2][1])
                - matrix[0][1] * (matrix[1][0] * matrix[2][2] - matrix[1][2] *
matrix[2][0])
                + matrix[0][2] * (matrix[1][0] * matrix[2][1] - matrix[1][1] *
matrix[2][0]);
        }
        public static int[][] replaceColumn(int[][] matrix, int[] constants, int col)
{
            int[][] newMatrix = new int[4][4];
            for (int i = 0; i < 4; i++) {
                for (int j = 0; j < 4; j++) {
                    newMatrix[i][j] = (j == col) ? constants[i] : matrix[i][j];
                }
```

```
            }
            return newMatrix;
        }
    }
```